Here's a complete explanation of **why Python is dynamically typed**:

🐍 WHAT IS TYPE SYSTEM FIRST?

Type System = Rules that define how a

programming language handles data types

Every variable stores some type of data:

→ Number (int, float)

→ Text (string)

→ True/False (boolean)

→ List, Dict, etc.

The question is → WHEN does the language

check what type a variable is?

📊 TWO TYPES OF TYPE SYSTEMS

```
┌──────────────────┬──────────────────┐
│ STATIC TYPING    │ DYNAMIC TYPING   │
├──────────────────┼──────────────────┤
│ Type checked at  │ Type checked at  │
│ COMPILE time     │ RUNTIME          │
├──────────────────┼──────────────────┤
│ Must declare     │ No need to       │
│ type explicitly  │ declare type     │
├──────────────────┼──────────────────┤
│ Java, C, C++,    │ Python, Ruby,    │
│ Swift, Kotlin    │ JavaScript, PHP  │
└──────────────────┴──────────────────┘
```

🔍 CORE REASON — WHY PYTHON IS DYNAMIC

In Python:

✅ Variables have NO fixed type

✅ Type is determined at RUNTIME

✅ Same variable can hold ANY type

✅ Type is attached to VALUE not VARIABLE

The Key Concept:

python

```
# In Python — variable is just a LABEL
# The VALUE carries the type, not the variable

x = 10        # x points to integer 10
x = "Hello"    # x now points to string
x = 3.14       # x now points to float
x = [1, 2, 3]  # x now points to a list

# Python allows all of this — NO ERROR!
# The variable 'x' has no fixed type
```

⚖️ **PYTHON vs JAVA COMPARISON**

Java (Static Typing):

java

```
// Must declare type explicitly
int x = 10;
x = "Hello";  // ❌ ERROR!
              // Cannot assign String to int
              // Caught at COMPILE time
```

Python (Dynamic Typing):

python

```
# No type declaration needed
x = 10
x = "Hello"   # ✅ Perfectly fine!
```

x = 3.14 # ✅ Still fine!

Type checked at RUNTIME

🔬 **HOW PYTHON HANDLES TYPES INTERNALLY**

In Python everything is an OBJECT

When you write:

x = 10

Python internally does:

1. Creates an integer object → value=10
2. Assigns memory address to x
3. x just POINTS to that object

When you write:

x = "Hello"

Python internally does:

1. Creates a string object → value="Hello"
2. Assigns NEW memory address to x
3. x now POINTS to string object
4. Old integer object gets garbage collected

Visual Representation:

x = 10

x ———→ [int object: 10]

x = "Hello"

x ———→ [str object: "Hello"]

[int object: 10] → deleted

x = [1,2,3]

x ———→ [list object: [1,2,3]]

[str object: "Hello"] → deleted

🧪 **PROOF WITH CODE EXAMPLES**

Example 1 — Type changes at runtime:

python

```
x = 10
print(type(x))    # <class 'int'>

x = "Python"
print(type(x))    # <class 'str'>

x = 3.14
print(type(x))    # <class 'float'>

x = True
print(type(x))    # <class 'bool'>

x = [1, 2, 3]
print(type(x))    # <class 'list'>

# Same variable — 5 different types!
# Python never complained once ✅
```

Example 2 — Type error caught at RUNTIME:

python

```
x = "Hello"
y = 10

result = x + y
# ❌ TypeError caught at RUNTIME
# "can only concatenate str (not int) to str"
```

```
# Python didn't know this was wrong
# until it actually ran this line
```

Example 3 — Function accepts any type:

python

```
def display(value):
    print(type(value), "→", value)

display(10)         # <class 'int'> → 10
display("Python")    # <class 'str'> → Python
display([1,2,3])     # <class 'list'> → [1,2,3]
display(True)        # <class 'bool'> → True

# Same function — works with ANY type!
# No type declaration needed
```

Example 4 — Duck Typing:

python

```
# "If it walks like a duck and quacks
#  like a duck, it's a duck"

# Python doesn't care about TYPE
# It only cares if the METHOD EXISTS

class Dog:
    def speak(self):
        return "Woof!"

class Cat:
    def speak(self):
        return "Meow!"

class Robot:
```

```
    def speak(self):
        return "Beep Boop!"

def make_sound(animal):
    print(animal.speak())  # Don't care about type
                           # Just needs speak() method

make_sound(Dog())    # Woof!
make_sound(Cat())    # Meow!
make_sound(Robot())  # Beep Boop!

# Python doesn't check the TYPE
# It checks if the METHOD exists at runtime
```

🔄 **DYNAMIC TYPING IN REAL SCENARIOS**

Scenario 1 — Data from API:

python

```
# You don't always know the type
# of data coming from an API

def process_data(data):
    if isinstance(data, int):
        return data * 2
    elif isinstance(data, str):
        return data.upper()
    elif isinstance(data, list):
        return len(data)

print(process_data(5))         # 10
print(process_data("hello"))    # HELLO
print(process_data([1,2,3,4]))  # 4
```

Scenario 2 — Selenium (your domain!):

python

```
# Dynamic typing helps in automation too

def get_element_value(element):
    value = element.get_attribute("value")
    # value could be string, number, None
    # Python handles it dynamically

    if value is None:
        return "No value found"
    return value

# Works regardless of what type
# the attribute returns!
```

✅ **ADVANTAGES OF DYNAMIC TYPING**

1. FASTER TO WRITE
   → No need to declare types
   → Less boilerplate code
   → Quicker prototyping

2. MORE FLEXIBLE
   → Functions work with multiple types
   → Variables can be reused
   → Less restrictive coding

3. EASIER TO LEARN
   → Beginners don't worry about types
   → Focus on logic not syntax
   → More readable code

4. GREAT FOR SCRIPTING

→ Automation scripts

→ Data processing

→ Rapid development

❌ **DISADVANTAGES OF DYNAMIC TYPING**

1. BUGS CAUGHT LATE

→ Type errors only at runtime

→ Not caught before execution

→ Can crash in production

2. HARDER TO DEBUG

→ Type confusion in large projects

→ Hard to track variable types

3. SLOWER PERFORMANCE

→ Type checking at runtime = overhead

→ Statically typed languages run faster

4. LESS IDE SUPPORT

→ IDE can't always predict type

→ Fewer autocomplete suggestions

🔧 **SOLUTION — TYPE HINTS (Python 3.5+)**

Python added OPTIONAL type hints

to get benefits of both worlds!

python

```
# Without type hints (pure dynamic)
def add(a, b):
    return a + b
```

```
# With type hints (optional static-like)
def add(a: int, b: int) -> int:
    return a + b

# Type hints are just HINTS
# Python still doesn't enforce them
# But tools like mypy can check them

name: str = "Rahul"
age: int = 25
scores: list[int] = [90, 85, 92]
```

📊 **SUMMARY TABLE**

Feature	Static (Java)	Dynamic (Python)
Type declaration	Required	Not needed
Type checking	Compile time	Runtime
Flexibility	Low	High
Speed	Faster	Slower
Bug detection	Early	Late
Code length	More	Less
Learning curve	Harder	Easier

💬 **INTERVIEW ANSWER**

Q: Why is Python dynamically typed?

A: Python is dynamically typed because
variable types are determined at RUNTIME,
not at compile time. In Python, a variable
is just a label that points to an object —

the type belongs to the object/value, not the variable itself. This means the same variable can hold different types at different points in the program.

For example:

x = 10 → integer

x = "hello" → string

x = 3.14 → float

Python allows this without any error. This makes Python flexible and easy to write but type errors are only caught at runtime, not before execution.

💡 **One line to remember: "In Python, variables don't have types — VALUES have types!"**